

PINN_heat_transfer_2d_solution

January 14, 2026

```
[1]: import imageio.v3 as imageio
      #Pawel Maczuga and Maciej Paszynski (2023)
      from typing import Callable

      import matplotlib.pyplot as plt
      import numpy as np
      import torch
      from torch import nn
      from matplotlib.animation import FuncAnimation
      # # # # # # # from google.colab import files
      import os
      from typing import Tuple

      device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
      device

      if os.path.basename(os.getcwd()) != "project1_heat":
          os.chdir("./project1_heat")
```

0.1 Parameters

```
[2]: LENGTH = 1.          # Domain size in x axis. Always starts at 0
      TOTAL_TIME = 1.      # Domain size in t axis. Always starts at 0
      N_POINTS = 64         # INCREASED from 15 to capture complex IC
      N_POINTS_PLOT = 150   # Number of points in single axis used in
      ↪plotting
      WEIGHT_RESIDUAL = 1.0 # Weight of residual part of loss function
      WEIGHT_INITIAL = 10.0 # INCREASED to prioritize initial shape
      WEIGHT_BOUNDARY = 1.0 # Weight of boundary part of loss function
      LAYERS = 4
      NEURONS_PER_LAYER = 80
      EPOCHS = 10000        # INCREASED from 2000 for convergence
      LEARNING_RATE = 0.002
```

$$1 \quad \frac{\partial u}{\partial t} - \frac{\partial^2 u}{\partial x^2} - \frac{\partial^2 u}{\partial y^2} = 0$$

Szukamy funkcji temperatury $[0, 1]^2 \times [0, T] \ni (x, y) \rightarrow u(x, y; t) \in \Re$

Warunek brzegowy $\frac{\partial u}{\partial n} = 0$ na całym brzegu

Stan początkowy

$$u_0 = u(x, y; 0) = \frac{\exp(-(7r)^2)}{2}$$

$r = \sqrt{(x - 0.5)^2 + (y - 0.5)^2}$ czyli $I(r < 1/4)$ oznacza że dla $r > 1/4$ to jest zero

$$LOSS_{PDE}(x, y, t) = (\partial u(x, y, t) / \partial t - \partial^2 u(x, y, t) / \partial x^2 - \partial^2 u(x, y, t) / \partial y^2)^2$$

$$LOSS_{BC} = (\partial u / \partial n)^2$$

czyli

$LOSS_{BC}(0, 0) - (1, 0) = (-\partial u / \partial x)^2$ dla x z przedziału $(0, 1)$ $y = 0$ $LOSS_{BC}(0, 1) - (1, 1) = (\partial u / \partial x)^2$ dla x z przedziału $(0, 1)$ $y = 1$ $LOSS_{BC}(0, 0) - (0, 1) = (-\partial u / \partial y)^2$ dla y z przedziału $(0, 1)$ $x = 0$ $LOSS_{BC}(1, 0) - (1, 1) = (\partial u / \partial y)^2$ dla y z przedziału $(0, 1)$ $x = 1$

$$LOSS_{INIT}(x, y; 0) = (u(x, y;) - u_0(x, y; 0))^2$$

1.1 Initial condition

```
[3]: import imageio.v3 as iio
import os

IMAGE_PATH = "initial_condition.png"
if not os.path.exists(IMAGE_PATH):
    IMAGE_PATH = "project1_heat/initial_condition.png"

if os.path.exists(IMAGE_PATH):
    print(f"Loading initial condition from {IMAGE_PATH}")
    img_data = iio.imread(IMAGE_PATH)
    if len(img_data.shape) > 2: img_data = img_data.mean(axis=-1)
    img_data = (img_data / 255.0) * 0.5
    INIT_TENSOR = torch.tensor(img_data, dtype=torch.float32).unsqueeze(0).
    ↪unsqueeze(0).to(device)
    print(f"Initial tensor shape: {INIT_TENSOR.shape}")
else:
    print("Warning: initial_condition.png not found. Using zero init.")
    INIT_TENSOR = torch.zeros(1, 1, 256, 256).to(device)

def initial_condition(x: torch.Tensor, y: torch.Tensor) -> torch.Tensor:
    grid_x = 2.0 * (x / LENGTH) - 1.0
    grid_y = 1.0 - 2.0 * (y / LENGTH)
    original_shape = x.shape
    x_flat = grid_x.view(1, 1, -1, 1)
    y_flat = grid_y.view(1, 1, -1, 1)
    grid = torch.cat([x_flat, y_flat], dim=-1)
    t_t = INIT_TENSOR.to(x.device)
    sampled = torch.nn.functional.grid_sample(t_t, grid, align_corners=True)
    return sampled.view(original_shape)
```

Loading initial condition from initial_condition.png
Initial tensor shape: torch.Size([1, 1, 256, 256])

1.2 PINN

```
[4]: class PINN(nn.Module):
    """Simple neural network accepting two features as input and returning a
    ↪single output

    In the context of PINNs, the neural network is used as universal function
    ↪approximator
    to approximate the solution of the differential equation
    """
    def __init__(self, num_hidden: int, dim_hidden: int, act=nn.Tanh()):

        super().__init__()

        self.layer_in = nn.Linear(3, dim_hidden)
        self.layer_out = nn.Linear(dim_hidden, 1)

        num_middle = num_hidden - 1
        self.middle_layers = nn.ModuleList(
            [nn.Linear(dim_hidden, dim_hidden) for _ in range(num_middle)]
        )
        self.act = act

    def forward(self, x, y, t):

        x_stack = torch.cat([x, y, t], dim=1)
        out = self.act(self.layer_in(x_stack))
        for layer in self.middle_layers:
            out = self.act(layer(out))
        logits = self.layer_out(out)

        return logits

    def device(self):
        return next(self.parameters()).device

def f(pinn: PINN, x: torch.Tensor, y: torch.Tensor, t: torch.Tensor) -> torch.
    ↪Tensor:
    """Compute the value of the approximate solution from the NN model"""
    return pinn(x, y, t)
```

```

def df(output: torch.Tensor, input: torch.Tensor, order: int = 1) -> torch.
↳Tensor:
    """Compute neural network derivative with respect to input features using
    ↳PyTorch autograd engine"""
    df_value = output
    for _ in range(order):
        df_value = torch.autograd.grad(
            df_value,
            input,
            grad_outputs=torch.ones_like(input),
            create_graph=True,
            retain_graph=True,
        )[0]

    return df_value

def dfdt(pinn: PINN, x: torch.Tensor, y: torch.Tensor, t: torch.Tensor, order:
↳int = 1):
    f_value = f(pinn, x, y, t)
    return df(f_value, t, order=order)

def dfdx(pinn: PINN, x: torch.Tensor, y: torch.Tensor, t: torch.Tensor, order:
↳int = 1):
    f_value = f(pinn, x, y, t)
    return df(f_value, x, order=order)

def dfdy(pinn: PINN, x: torch.Tensor, y: torch.Tensor, t: torch.Tensor, order:
↳int = 1):
    f_value = f(pinn, x, y, t)
    return df(f_value, y, order=order)

```

1.3 Loss function

```

[5]: def get_boundary_points(x_domain, y_domain, t_domain, n_points, device = torch.
↳device("cpu"), requires_grad=True):
    """
        .+-----+
        .' /      .' /
    +---+---+' /
    /    /    /
    y / ,+---+---+
    /.'      /.' t
    +-----+'
        x
    """

```

```

"""
x_linspace = torch.linspace(x_domain[0], x_domain[1], n_points)
y_linspace = torch.linspace(y_domain[0], y_domain[1], n_points)
t_linspace = torch.linspace(t_domain[0], t_domain[1], n_points)

x_grid, t_grid = torch.meshgrid( x_linspace, t_linspace, indexing="ij")
y_grid, _      = torch.meshgrid( y_linspace, t_linspace, indexing="ij")

x_grid = x_grid.reshape(-1, 1).to(device)
x_grid.requires_grad = requires_grad
y_grid = y_grid.reshape(-1, 1).to(device)
y_grid.requires_grad = requires_grad
t_grid = t_grid.reshape(-1, 1).to(device)
t_grid.requires_grad = requires_grad

x0 = torch.full_like(t_grid, x_domain[0], requires_grad=requires_grad)
x1 = torch.full_like(t_grid, x_domain[1], requires_grad=requires_grad)
y0 = torch.full_like(t_grid, y_domain[0], requires_grad=requires_grad)
y1 = torch.full_like(t_grid, y_domain[1], requires_grad=requires_grad)

down   = (x_grid, y0,    t_grid)
up     = (x_grid, y1,    t_grid)
left   = (x0,    y_grid, t_grid)
right  = (x1,    y_grid, t_grid)

return down, up, left, right

```

```

[6]: def get_initial_points(x_domain, y_domain, t_domain, n_points, device = torch.
    ↪device("cpu"), requires_grad=True):
    x_linspace = torch.linspace(x_domain[0], x_domain[1], n_points)
    y_linspace = torch.linspace(y_domain[0], y_domain[1], n_points)
    x_grid, y_grid = torch.meshgrid( x_linspace, y_linspace, indexing="ij")
    x_grid = x_grid.reshape(-1, 1).to(device)
    x_grid.requires_grad = requires_grad
    y_grid = y_grid.reshape(-1, 1).to(device)
    y_grid.requires_grad = requires_grad
    t0 = torch.full_like(x_grid, t_domain[0], requires_grad=requires_grad)
    return (x_grid, y_grid, t0)

```

```

[7]: def get_interior_points(x_domain, y_domain, t_domain, n_points, device = torch.
    ↪device("cpu"), requires_grad=True):
    x_raw = torch.linspace(x_domain[0], x_domain[1], steps=n_points,
    ↪requires_grad=requires_grad)
    y_raw = torch.linspace(y_domain[0], y_domain[1], steps=n_points,
    ↪requires_grad=requires_grad)
    t_raw = torch.linspace(t_domain[0], t_domain[1], steps=n_points,
    ↪requires_grad=requires_grad)

```

```

grids = torch.meshgrid(x_raw, y_raw, t_raw, indexing="ij")

x = grids[0].reshape(-1, 1).to(device)
y = grids[1].reshape(-1, 1).to(device)
t = grids[2].reshape(-1, 1).to(device)

return x, y, t

```

```

[8]: class Loss:
    def __init__(
        self,
        x_domain: Tuple[float, float],
        y_domain: Tuple[float, float],
        t_domain: Tuple[float, float],
        n_points: int,
        initial_condition: Callable,
        weight_r: float = 1.0,
        weight_b: float = 1.0,
        weight_i: float = 1.0,
        verbose: bool = False,
    ):
        self.x_domain = x_domain
        self.y_domain = y_domain
        self.t_domain = t_domain
        self.n_points = n_points
        self.initial_condition = initial_condition
        self.weight_r = weight_r
        self.weight_b = weight_b
        self.weight_i = weight_i

    def residual_loss(self, pinn: PINN):
        x, y, t = get_interior_points(self.x_domain, self.y_domain, self.
↪t_domain, self.n_points, pinn.device())
        loss = dfdt(pinn, x, y, t) - dfdx(pinn, x, y, t, order=2) - dfdy(pinn,
↪x, y, t, order=2)
        return loss.pow(2).mean()

    def initial_loss(self, pinn: PINN):
        x, y, t = get_initial_points(self.x_domain, self.y_domain, self.
↪t_domain, self.n_points, pinn.device())
        pinn_init = self.initial_condition(x, y)
        loss = f(pinn, x, y, t) - pinn_init
        return loss.pow(2).mean()

    def boundary_loss(self, pinn: PINN):
        down, up, left, right = get_boundary_points(self.x_domain, self.
↪y_domain, self.t_domain, self.n_points, pinn.device())

```

```

x_down, y_down, t_down = down
x_up, y_up, t_up = up
x_left, y_left, t_left = left
x_right, y_right, t_right = right

loss_down = dfdy( pinn, x_down, y_down, t_down )
loss_up = dfdy( pinn, x_up, y_up, t_up )
loss_left = dfdx( pinn, x_left, y_left, t_left )
loss_right = dfdx( pinn, x_right, y_right, t_right )

return loss_down.pow(2).mean() + \
       loss_up.pow(2).mean() + \
       loss_left.pow(2).mean() + \
       loss_right.pow(2).mean()

def verbose(self, pinn: PINN):
    """
    Returns all parts of the loss function

    Not used during training! Only for checking the results later.
    """
    residual_loss = self.residual_loss(pinn)
    initial_loss = self.initial_loss(pinn)
    boundary_loss = self.boundary_loss(pinn)

    final_loss = \
        self.weight_r * residual_loss + \
        self.weight_i * initial_loss + \
        self.weight_b * boundary_loss

    return final_loss, residual_loss, initial_loss, boundary_loss

def __call__(self, pinn: PINN):
    """
    Allows you to use instance of this class as if it was a function:

    ...
    >>> loss = Loss(*some_args)
    >>> calculated_loss = loss(pinn)
    ...
    """
    return self.verbose(pinn)[0]

```

1.4 Train function

```
[9]: def train_model(
    nn_approximator: PINN,
    loss_fn: Callable,
    learning_rate: int = 0.01,
    max_epochs: int = 1_000
) -> PINN:

    optimizer = torch.optim.Adam(nn_approximator.parameters(), lr=learning_rate)
    loss_values = []
    for epoch in range(max_epochs):

        try:

            loss: torch.Tensor = loss_fn(nn_approximator)
            optimizer.zero_grad()
            loss.backward()
            optimizer.step()

            loss_values.append(loss.item())
            if (epoch + 1) % 1000 == 0:
                print(f"Epoch: {epoch + 1} - Loss: {float(loss):>7f}")

        except KeyboardInterrupt:
            break

    return nn_approximator, np.array(loss_values)
```

1.5 Plotting functions

```
[10]: def plot_solution(pinn: PINN, x: torch.Tensor, t: torch.Tensor, figsize=(8, 6),
    ↪dpi=100):

    fig, ax = plt.subplots(figsize=figsize, dpi=dpi)
    x_raw = torch.unique(x).reshape(-1, 1)
    t_raw = torch.unique(t)

    def animate(i):

        if not i % 10 == 0:
            t_partial = torch.ones_like(x_raw) * t_raw[i]
            f_final = f(pinn, x_raw, t_partial)
            ax.clear()
            ax.plot(
                x_raw.detach().numpy(), f_final.detach().numpy(), label=f"Time_
                ↪{float(t[i])}")
```



```

        )
        ax.set_ylim(-1, 1)
        ax.legend()

    n_frames = t_raw.shape[0]
    return FuncAnimation(fig, animate, frames=n_frames, interval=100,
↪repeat=False)

def plot_color(z: torch.Tensor, x: torch.Tensor, y: torch.Tensor, n_points_x,
↪n_points_t, title, figsize=(8, 6), dpi=100, cmap="viridis", vmin=None,
↪vmax=None):
    fig, ax = plt.subplots(figsize=figsize, dpi=dpi)
    z_raw = z.detach().cpu().numpy()
    x_raw = x.detach().cpu().numpy()
    y_raw = y.detach().cpu().numpy()
    X = x_raw.reshape(n_points_x, n_points_t)
    Y = y_raw.reshape(n_points_x, n_points_t)
    Z = z_raw.reshape(n_points_x, n_points_t)
    ax.set_title(title)
    ax.set_xlabel("x")
    ax.set_ylabel("y")
    c = ax.pcolormesh(X, Y, Z, cmap=cmap, vmin=vmin, vmax=vmax)
    fig.colorbar(c, ax=ax)

    return fig

def plot_3D(z: torch.Tensor, x: torch.Tensor, y: torch.Tensor, n_points_x,
↪n_points_t, title, figsize=(8, 6), dpi=100, limit=0.6):
    fig = plt.figure(figsize=figsize)
    ax = fig.add_subplot(projection='3d')
    z_raw = z.detach().cpu().numpy()
    x_raw = x.detach().cpu().numpy()
    y_raw = y.detach().cpu().numpy()
    X = x_raw.reshape(n_points_x, n_points_t)
    Y = y_raw.reshape(n_points_x, n_points_t)
    Z = z_raw.reshape(n_points_x, n_points_t)
    ax.set_title(title)
    ax.set_xlabel("x")
    ax.set_ylabel("y")
    ax.axes.set_zlim3d(bottom=-limit, top=limit)

    c = ax.plot_surface(X, Y, Z)

    return fig

def running_average(y, window=100):
    cumsum = np.cumsum(np.insert(y, 0, 0))

```

```
return (cumsum>window:] - cumsum[:>window]) / float(window)
```

2 Running code

2.1 Train data

```
[11]: pinn = PINN(LAYERS, NEURONS_PER_LAYER, act=nn.Tanh()).to(device)

x_domain = [0.0, LENGTH]
y_domain = [0.0, LENGTH]
t_domain = [0.0, TOTAL_TIME]

# train the PINN
loss_fn = Loss(
    x_domain,
    y_domain,
    t_domain,
    N_POINTS,
    initial_condition,
    WEIGHT_RESIDUAL,
    WEIGHT_INITIAL,
    WEIGHT_BOUNDARY
)

pinn_trained, loss_values = train_model(
    pinn, loss_fn=loss_fn, learning_rate=LEARNING_RATE, max_epochs=EPOCHS)
```

```
C:\Users\lopma\Education\Studies\sem9\continuous-process-simulation-and-
adaptive-algorithms\pinn-labs\.venv\Lib\site-
packages\torch\autograd\graph.py:823: UserWarning: Attempting to run cuBLAS, but
there was no current CUDA context! Attempting to set the primary context...
(Triggered internally at C:\actions-runner\work\pytorch\pytorch\pytorch\aten\sr
c\ATen\cuda\CublasHandlePool.cpp:180.)
```

```
    return Variable._execution_engine.run_backward( # Calls into the C++ engine
to run the backward pass
```

```
Epoch: 1000 - Loss: 0.035932
```

```
Epoch: 2000 - Loss: 0.035593
```

```
Epoch: 3000 - Loss: 0.034227
```

```
Epoch: 4000 - Loss: 0.033452
```

```
Epoch: 5000 - Loss: 0.032854
```

```
Epoch: 6000 - Loss: 0.035641
```

```
Epoch: 7000 - Loss: 0.032101
```

```
Epoch: 8000 - Loss: 0.025507
```

Epoch: 9000 - Loss: 0.022416

Epoch: 10000 - Loss: 0.021128

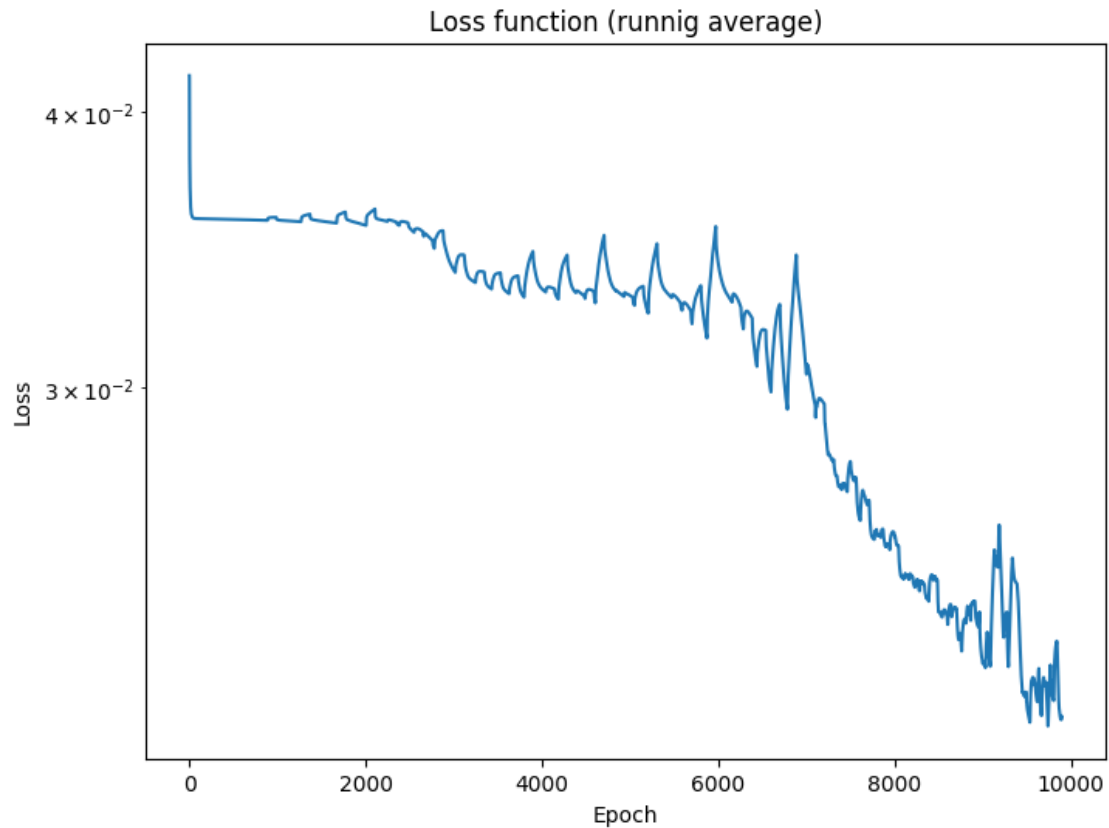
```
[12]: pinn = pinn.cpu()
```

```
[13]: losses = loss_fn.verbose(pinn)
print(f'Total loss: \t{losses[0]:.5f}      ({losses[0]:.3E})')
print(f'Interior loss: \t{losses[1]:.5f}      ({losses[1]:.3E})')
print(f'Initial loss: \t{losses[2]:.5f}      ({losses[2]:.3E})')
print(f'Bondary loss: \t{losses[3]:.5f}      ({losses[3]:.3E})')
```

Total loss:	0.01976	(1.976E-02)
Interior loss:	0.00126	(1.263E-03)
Initial loss:	0.01830	(1.830E-02)
Bondary loss:	0.00002	(1.973E-05)

3 Plotting

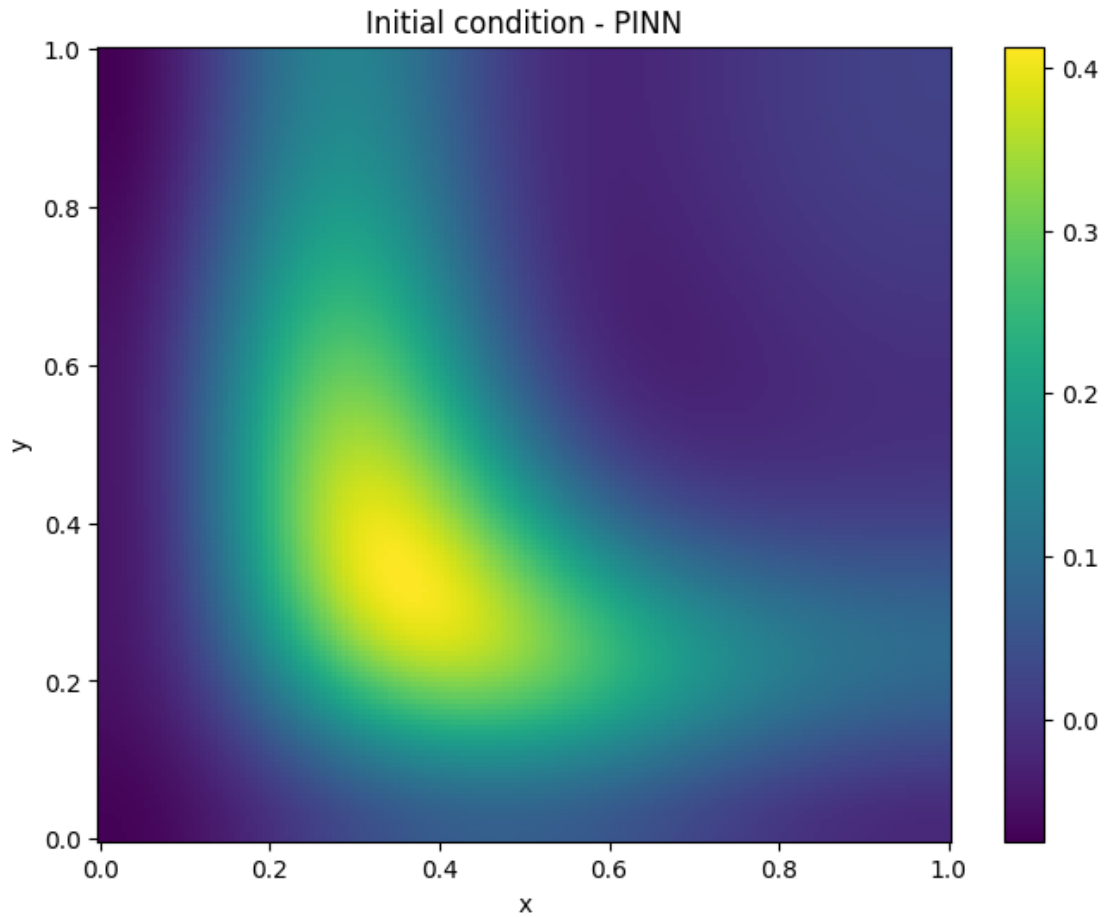
```
[14]: # Loss function
average_loss = running_average(loss_values, window=100)
fig, ax = plt.subplots(figsize=(8, 6), dpi=100)
ax.set_title("Loss function (running average)")
ax.set_xlabel("Epoch")
ax.set_ylabel("Loss")
ax.plot(average_loss)
ax.set_yscale('log')
```



```
[15]: # Initial condition
```

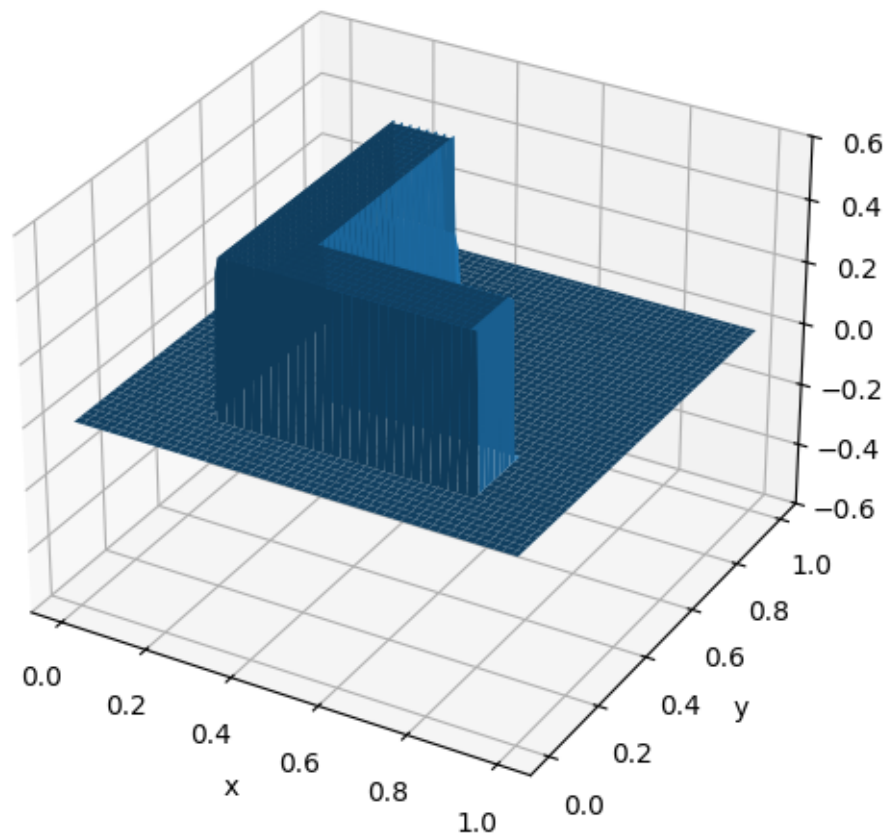
```
[16]: x, y, _ = get_initial_points(x_domain, y_domain, t_domain, N_POINTS_PLOT,
    ↪requires_grad=False)
z = initial_condition(x, y)
# fig = plot_color(z, x, y, N_POINTS_PLOT, N_POINTS_PLOT, "Initial condition -
    ↪exact")
```

```
[17]: t_value = 0.0
t = torch.full_like(x, t_value)
z = pinn(x, y, t)
fig = plot_color(z, x, y, N_POINTS_PLOT, N_POINTS_PLOT, "Initial condition -
    ↪PINN")
```



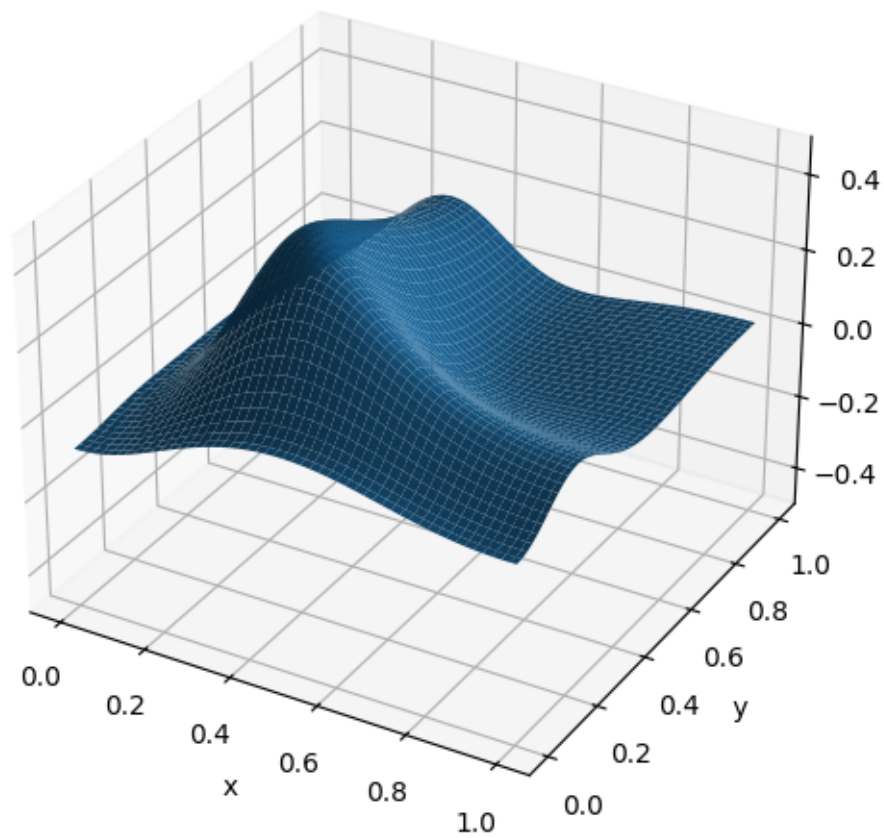
```
[18]: x, y, _ = get_initial_points(x_domain, y_domain, t_domain, N_POINTS_PLOT,
    ↳requires_grad=False)
z = initial_condition(x, y)
fig = plot_3D(z, x, y, N_POINTS_PLOT, N_POINTS_PLOT, "Initial condition -
    ↳exact", limit=0.6)
```

Initial condition - exact



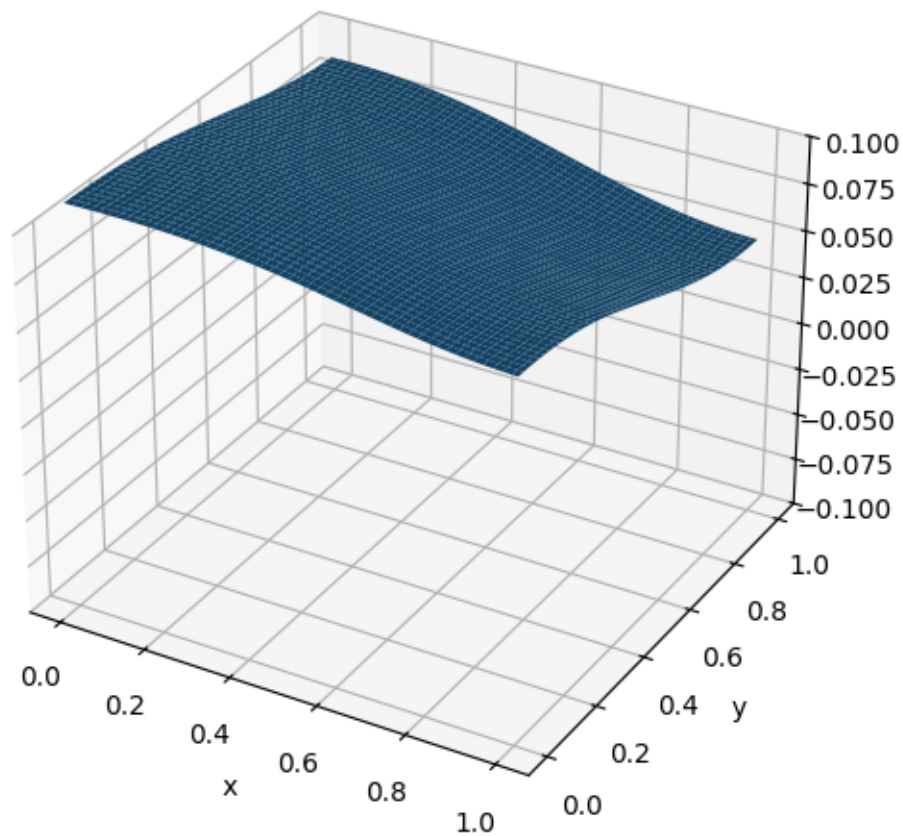
```
[19]: t_value = 0.0
      t = torch.full_like(x, t_value)
      z = pinn(x, y, t)
      fig = plot_3D(z, x, y, N_POINTS_PLOT, N_POINTS_PLOT, "Initial condition - PINN", limit=0.5)
```

Initial condition - PINN



```
[20]: t_value = 0.1
      t = torch.full_like(x, t_value)
      z = pinn(x, y, t)
      fig = plot_3D(z, x, y, N_POINTS_PLOT, N_POINTS_PLOT, f"PINN for t = {t_value}",
                    ↪ limit=0.1)
```

PINN for $t = 0.1$



```
[21]: # diff = []
# ts = torch.linspace(0, 1, 10)
# for t_value in ts:
#     t = torch.full_like(x, t_value)
#     z = pinn(x, y, t)
#     z = pinn(x, y, t)
#     # # # # # z_exact = exact(x, y, t) # Removed because no analytic
#     ↪solution for bitmap
#     z_diff = (z-z_exact).pow(2).mean() # Removed because no analytic
#     ↪solution for bitmap
#     diff.append(z_diff.detach())
#
# plt.plot(ts.detach().numpy(), diff)
```

```
[22]: import os
import torch
import matplotlib.pyplot as plt
```



```

import imageio.v2 as iio
import numpy as np

# Ensure output directories exist
os.makedirs('img', exist_ok=True)
os.makedirs('output', exist_ok=True)

print("Starting GIF generation with DYNAMIC SCALING...")

# Define time values for animation
n_frames = 50
time_values = torch.linspace(0, 0.2, n_frames).to(device)

# Store Z values to find global min/max
print("Pre-computing frames to determine scale...")
frames_z = []
for t_val in time_values:
    t_tensor = torch.full_like(x, t_val)
    with torch.no_grad():
        z = pinn(x, y, t_tensor)
        frames_z.append(z)

# Compute global min/max across ALL frames
# We want the scale to be consistent across the animation, but fitting the
↳ actual data range.
all_min = min([z.min().item() for z in frames_z])
all_max = max([z.max().item() for z in frames_z])

print(f"Global Range determined: Min={all_min:.4f}, Max={all_max:.4f}")

# Add a tiny buffer if min == max to avoid division by zero errors in plotting
if all_max == all_min:
    all_max += 1e-6

# Generate plots
for i, z in enumerate(frames_z):
    t_val = time_values[i].item()

    # Plot using the global scale
    fig = plot_color(z, x, y, N_POINTS_PLOT, N_POINTS_PLOT, f"Time: {t_val:.
↳ 2f}", vmin=all_min, vmax=all_max)

    # Save frame
    frame_path = f'img/frame_{i:03d}.png'
    fig.savefig(frame_path)
    plt.close(fig)

```

```

print("Frames generated.")

# Compile GIF
frames = []
for i in range(n_frames):
    frame_path = f'img/frame_{i:03d}.png'
    if os.path.exists(frame_path):
        frames.append(iio.imread(frame_path))

if frames:
    gif_path = 'output/heat_transfer.gif'
    iio.mimsave(gif_path, frames, duration=0.1)
    print(f"GIF saved to {gif_path}")
else:
    print("No frames found, GIF not created.")

```

Starting GIF generation with DYNAMIC SCALING...

Pre-computing frames to determine scale...

Global Range determined: Min=-0.0750, Max=0.4129

Frames generated.

GIF saved to output/heat_transfer.gif